

รายงาน Lab 8

ส่วนที่ 1: หลักการออกแบบ

1.1 SOLID Principles และความเกี่ยวข้องกับ Entity

โปรเจกต์นี้ออกแบบ Entity, Repository, Service และ Controller โดยยึดหลัก SOLID ทั้ง 5 ข้อ ดังในตารางต่อไปนี้:

Principle	การประยุกต์ใช้ในโปรเจกต์
S — SRP	แยก Entity ตามความรับผิดชอบ: Product ดูแลข้อมูลหลักของสินค้า, ProductDetail ดูแลข้อมูลเสริมที่ไม่ได้ใช้ทุก query, Review ดูแลข้อมูลรีวิวเท่านั้น ProductService ก็รับผิดชอบเฉพาะ business logic ส่วน ProductController รับผิดชอบเฉพาะการรับ-ส่ง HTTP
O — OCP	เพิ่มความสามารถใหม่โดยไม่ต้องแก้ไขโค้ดเดิม เช่น เพิ่ม Entity Review เข้ามาโดยไม่ต้องแก้ field เดิมของ Product (มีแค่ List<Review> ที่เปิดรับการขยายไว้) และเพิ่มส่วนลดชนิดใหม่ได้โดยสร้าง class ที่ implement DiscountStrategy เพิ่มโดยไม่ต้องแก้ DiscountContext หรือ Product เดิม
L — LSP	ProductRepository, ProductDetailRepository, ReviewRepository ทุกตัว extends JpaRepository<T, ID> และถูกใช้งานผ่าน type ของ interface เดียวกัน ทำให้ Service เรียกใช้ repository ใดๆ ที่ implement สัญญาเดียวกันนี้แทนกันได้โดยพฤติกรรมยังถูกต้องเหมือนเดิม เช่นเดียวกับ DiscountStrategy ที่ implementation ทั้ง 3 ตัว (NoDiscount, Member, Seasonal) ใช้แทนกันได้ผ่าน DiscountContext
I — ISP	แยก Repository ออกเป็น 3 interface เล็กๆ ตามหน้าที่ (ProductRepository, ProductDetailRepository, ReviewRepository) แทนที่จะรวมเป็น interface ใหญ่ตัวเดียว และ DiscountStrategy ก็มีเพียง method เดียวคือ applyDiscount(...) ไม่บังคับให้ implementation ต้องมี method อื่นที่ไม่เกี่ยวข้อง
D — DIP	ProductService ขึ้นกับ ProductRepository ในฐานะ interface (abstraction) ไม่ใช่ implementation จริง และรับเข้ามาผ่าน Constructor Injection เช่นเดียวกับ ProductController ที่ขึ้นกับ ProductService ผ่าน constructor และ Product.getDiscountedPrice() ที่ขึ้นกับ DiscountStrategy (interface) ผ่าน DiscountContext แทนที่จะเขียน if/else ตรวจสอบ discountType ในตัวเอง

การแยก Product / ProductDetail / Review ออกเป็นคณะ Entity ไม่ได้ทำเพราะ JPA บังคับ แต่เป็นผลจากการยึด SRP ตั้งแต่ขั้นออกแบบฐานข้อมูล ส่วนการใช้ Interface (Repository และ DiscountStrategy) ร่วมกับ Constructor Injection คือกลไกหลักที่ทำให้ DIP, LSP และ ISP เกิดขึ้นจริงในโค้ด ไม่ใช่แค่แนวคิดลอยๆ

1.2 ความแตกต่างของ 1:1 กับ 1:N — ใช้กรณีไหนควรใช้แบบไหน

ความสัมพันธ์ทั้งสองแบบต่างกันว่า "จำนวนแถวของตารางฝั่งตรงข้ามที่จับคู่ได้" และส่งผลโดยตรงต่อตำแหน่งที่ Foreign Key ควรอยู่:

หัวข้อ	One-to-One (1:1)	One-to-Many (1:N)
ความหมาย	1 แถวของตารางหลัก จับคู่กับอีกตารางได้เพียง 1 แถวเท่านั้น	1 แถวของตารางหลัก จับคู่กับอีกตารางได้หลายแถว
ตัวอย่างใน Lab	Product ↔ ProductDetail	Product → Review (หลายรีวิวต่อสินค้า)
ตำแหน่ง FK	ฝั่ง Owner (Product เก็บ detail_id)	ฝั่ง Many เสมอ (Review เก็บ product_id)

หัวข้อ	One-to-One (1:1)	One-to-Many (1:N)
Annotation ฝั่งเจ้าของ	@OneToOne + @JoinColumn	@OneToMany(mappedBy="product")
Annotation ฝั่งตรงข้าม	@OneToOne(mappedBy="detail")	@ManyToOne + @JoinColumn(name="product_id")
ควรใช้เมื่อ...	ข้อมูลเสริมที่ไม่จำเป็นต้องใช้ทุกครั้ง แยกออกเพื่อ SRP และลดขนาด query หลัก	ข้อมูล child ที่มีจำนวนไม่จำกัด/เพิ่มได้เรื่อยๆ โดยไม่กระทบโครงสร้างตารางหลัก (OCP)

หลักการเลือกใช้: ถ้าข้อมูลเป็น “ส่วนขยายของสิ่งเดียวกัน” ที่มีอยู่แค่ชุดเดียวต่อ 1 แถวหลัก และบางครั้งไม่จำเป็นต้องดึงมาด้วยทุกครั้ง (เช่น ProductDetail มีรายละเอียด/น้ำหนัก/ขนาดที่ไม่ต้องใช้ตอนแสดงรายการสินค้าแบบย่อ) ให้แยกเป็น 1:1 เพื่อคง SRP และลดขนาดของ query หลัก แต่ถ้าข้อมูลเป็น “รายการย่อยที่เพิ่มได้เรื่อยๆ ไม่จำกัดจำนวน” และแต่ละรายการเป็นเหตุการณ์/บันทึกอิสระ (เช่น Review ที่ลูกค้าคนไหนก็เขียนได้ไม่จำกัดจำนวนครั้ง) ต้องใช้ 1:N เพราะ 1:1 ไม่สามารถรองรับได้ตั้งแต่ระดับ schema

1.3 Strategy Pattern สำหรับคำนวณส่วนลด

ฟิลด์ discountType ของ Product เก็บค่าเป็น String (NONE / MEMBER / SEASONAL) แต่ตรรกะการคำนวณราคาไม่ได้เขียนเป็น if/else อยู่ใน Entity โดยตรง เพราะจะทำให้ Product ผูกติดกับกฎทางธุรกิจของส่วนลด และทุกครั้งที่เพิ่มส่วนลดชนิดใหม่ต้องแก้ไข Entity ซ้ำ (ขัดกับ OCP) จึงแยกออกเป็น Strategy Pattern ที่ประกอบด้วย 3 ส่วน:

- DiscountStrategy — interface กลาง มี method เดียวคือ applyDiscount(double originalPrice)
- Concrete Strategy — NoDiscountStrategy (0%), MemberDiscountStrategy (-10%), SeasonalSaleStrategy (-20%)
แต่ละคลาสรู้วิธีคำนวณของตัวเองเท่านั้น
- DiscountContext — คลาสกลางที่ถือ DiscountStrategy (ผ่าน constructor) แล้วเรียก calculatePrice(...) โดยไม่สนใจว่าภายในเป็น strategy ตัวไหน

เมื่อหน้าเว็บเรียก product.getDiscountedPrice() (ใช้ใน list.html ตอนแสดงราคาสุทธิ) เมธอดนี้จะ resolve string discountType ให้เป็น DiscountStrategy ที่ตรงกัน สร้าง DiscountContext แล้วคำนวณราคาให้ ผลคือ ถ้าต้องการเพิ่มส่วนลดใหม่ เช่น ส่วนลดวันเกิด เพียงสร้างคลาสใหม่ implements DiscountStrategy แล้วเพิ่มเข้า resolveStrategy(...) โดยไม่ต้องแก้ไขโค้ดคำนวณเดิมเลย

1.4 Execution Flow: ตั้งแต่ HTTP Request ถึง Database

ตัวอย่างการเพิ่มสินค้าใหม่ (Create) ผ่านฟอร์ม add.html มีลำดับการทำงานดังนี้:

1. ผู้ใช้กรอกฟอร์ม add.html แล้วกด “บันทึกสินค้า” — Browser ส่ง POST /products/save พร้อมข้อมูล name, price, detail.*, reviews[0].* แบบ form-urlencoded
2. Spring MVC's DispatcherServlet รับ request แล้วจับคู่กับเมธอด save(...) ใน ProductController ตาม @PostMapping("/save")
3. Spring ทำ Data Binding อัตโนมัติ แปลงข้อมูลฟอร์มเป็น object Product (รวมถึงสร้าง ProductDetail และ Review ซ่อนอยู่ข้างในผ่าน property path เช่น detail.warranty, reviews[0].reviewer)
4. Controller ไม่ประมวลผลเอง แต่ส่งต่อ object Product ไปยัง productService.saveNewProduct(product) ทันที (SRP: Controller ทำหน้าที่แค่ routing)
5. ProductService ทำ business logic: ผูก back-reference ของ ProductDetail และ Review เข้ากับ Product (setDetailWithBackReference, cleanAndLinkReviews) แล้วเรียก productRepository.save(product)

นายกานดิทัต นามสุดตา 673380392-1 Sec 3

- 6. ProductRepository (Spring Data JPA) แปลง save(...) เป็นคำสั่ง Hibernate/JPA เพื่อ INSERT ลงตาราง products และอาศัย cascade = CascadeType.ALL ที่กำหนดไว้บน field detail และ reviews ทำให้ Hibernate INSERT ลง product_details และ reviews ต่อในธุรกรรมเดียวกันโดยอัตโนมัติ พร้อมเติมค่า FK (detail_id ที่ products, product_id ที่ reviews) ให้เอง
- 7. เมื่อ transaction สำเร็จ Controller จะเซต flash attribute ข้อความแจ้งเตือน แล้ว redirect กลับไปที่ GET /products
- 8. Request ใหม่ (GET /products) วิ่งเข้า list(...) → productService.getAllProducts() → productRepository.findAll() → Hibernate ยิง SELECT พร้อม JOIN ตาราง product_details และ reviews (เพราะตั้ง fetch = FetchType.EAGER) แล้วส่งต่อ List<Product> กลับไปที่ Controller
- 9. Controller เติมข้อมูลลง Model แล้วส่งชื่อ View "products/list" ให้ Thymeleaf render เป็น HTML กลับไปแสดงผลที่ Browser ผู้ใช้

ส่วนที่ 2: Code และคำอธิบาย

2.1 Entity ทั้ง 3 ตัว

Product.java

Entity หลักของระบบ เป็นเจ้าของ (owning side) ของทั้งความสัมพันธ์ 1:1 กับ ProductDetail และ 1:N กับ Review:

```
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String category;
    private String brand;
    private Integer stock;
    private Double price;

    // ค่าที่เป็นไปได้: NONE, MEMBER, SEASONAL
    private String discountType;

    // 1:1 กับ ProductDetail – Product เป็นเจ้าของ FK (detail_id)
    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "detail_id", referencedColumnName = "id")
    private ProductDetail detail;

    // 1:N กับ Review – Product คือฝั่ง "1"
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL,
        orphanRemoval = true, fetch = FetchType.EAGER)
    private List<Review> reviews = new ArrayList<>();

    @Transient
    public Double getDiscountedPrice() {
        if (price == null) return null;
        DiscountStrategy strategy = resolveStrategy(discountType);
        DiscountContext context = new DiscountContext(strategy);
        return context.calculatePrice(price);
    }

    public void addReview(Review review) {
        reviews.add(review);
        review.setProduct(this);
    }

    public void setDetailWithBackReference(ProductDetail detail) {
        this.detail = detail;
        if (detail != null) detail.setProduct(this);
    }

    // ... getters / setters
}
```

Annotation / Field	คำอธิบาย
@Entity / @Table	ประกาศว่าคลาสนี้คือ JPA Entity และ map ไปยังตาราง products ในฐานข้อมูล
@Id / @GeneratedValue	กำหนด Primary Key และให้ฐานข้อมูลสร้างค่า id อัตโนมัติแบบ IDENTITY (auto-increment)
detail: @OneToOne + @JoinColumn("detail_id")	Product เป็นฝั่งเจ้าของความสัมพันธ์ 1:1 จึงเป็นฝั่งที่มีคอลัมน์ FK (detail_id) อยู่จริงในตาราง products; cascade = ALL ทำให้ INSERT/UPDATE/DELETE ที่ Product ส่งผลไปถึง ProductDetail ที่ผูกอยู่ด้วยอัตโนมัติ
reviews: @OneToMany(mappedBy="product")	Product เป็นฝั่ง "1" ของความสัมพันธ์ 1:N โดย mappedBy ระบุว่า FK (product_id) ถูกจัดการอยู่ที่ field "product" ใน Review แล้ว ฝั่งนี้จึงไม่มีคอลัมน์ FK เพิ่มในตาราง products; orphanRemoval = true ทำให้ Review ที่ถูกเอาออกจาก list จะถูกลบออกจากฐานข้อมูลด้วย
getDiscountedPrice() – @Transient	เมธอดคำนวณราคาหลังหักส่วนลดผ่าน Strategy Pattern ทำเครื่องหมาย @Transient เพื่อบอก JPA ว่าไม่ใช่คอลัมน์ในฐานข้อมูล เป็นค่าที่คำนวณสดทุกครั้งที่เรียกใช้เท่านั้น

ProductDetail.java

Entity ฝั่งข้อมูลเสริมของสินค้า เป็นฝั่ง inverse (ผู้ถูกอ้างอิง) ของความสัมพันธ์ 1:1:

```
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 1000)
    private String description;
    private String warranty;
    private Double weight;
    private String dimensions;
    private String manufacturedCountry;

    // ฝั่ง inverse ของ 1:1 – ไม่ใช่ FK จริง
    @OneToOne(mappedBy = "detail")
    private Product product;

    // ... constructors, getters / setters
}
```

Annotation / Field	คำอธิบาย
@OneToOne(mappedBy = "detail")	ระบุว่า field นี้เป็นฝั่ง inverse ของความสัมพันธ์ — ไม่ได้ถือ FK จริง (ไม่มีคอลัมน์ product_id ในตาราง product_details) แต่ mappedBy ชี้กลับไปยัง field ชื่อ "detail" ที่อยู่ใน Product ซึ่งเป็นฝั่งเจ้าของจริง
description / warranty / weight / dimensions / manufacturedCountry	ฟิลด์ข้อมูลเสริมของสินค้าที่ไม่จำเป็นต้องดึงมาทุกครั้ง query รายการสินค้า จึงถูกแยกออกจาก Product ตามหลัก SRP

Review.java

Entity ฝั่ง "Many" ของความสัมพันธ์ 1:N เป็นผู้ถือ Foreign Key จริง:

```
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String reviewer;
    private Integer rating;

    @Column(length = 1000)
    private String comment;
    private LocalDate reviewDate;

    // ฝั่ง Many ถือ FK เสมอ
    @ManyToOne
    @JoinColumn(name = "product_id")
    private Product product;

    // ... constructors, getters / setters
}
```

Annotation / Field	คำอธิบาย
product: @ManyToOne + @JoinColumn("product_id")	Review เป็นฝั่ง Many จึงเป็นฝั่งที่มีคอลลัมน์ FK จริงในตาราง (product_id) เพราะ Review หนึ่งรายการเป็นของ Product เดียวเท่านั้น แต่ Product หนึ่งชิ้นมี Review ได้หลายรายการ — นี่คือกฎที่ FK ต้องอยู่ฝั่ง Many เสมอในความสัมพันธ์ 1:N
reviewer / rating / comment / reviewDate	ข้อมูลของรีวิวแต่ละรายการ ไม่เกี่ยวข้องกับข้อมูลหลักของ Product จึงแยก Entity ตาม SRP เช่นเดียวกับ ProductDetail

2.2 Service และ Controller พร้อม Constructor Injection

ทั้ง ProductService และ ProductController รับ dependency ผ่าน constructor (Constructor Injection) แทนการใช้ @Autowired บน field โดยตรง ซึ่งมีข้อดีคือทำให้ dependency ทุกตัวถูกประกาศชัดเจนใน constructor, บังคับให้ dependency เป็น final (immutable หลังสร้าง object), และทดสอบหน่วย (unit test) ได้ง่ายเพราะสามารถส่ง mock ของ ProductRepository / ProductService เข้าไปแทนได้โดยไม่ต้องพึ่ง Spring container

ProductService.java

```
@Service
public class ProductService {

    private final ProductRepository productRepository;

    // Constructor Injection
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
}
```

```

public Product saveNewProduct(Product product) {
    linkDetail(product);
    cleanAndLinkReviews(product);
    return productRepository.save(product);
}

public Product updateProduct(Long id, Product formProduct) {
    Product existing = productRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException("ไม่พบสินค้ารหัส " + id));
    existing.setName(formProduct.getName());
    existing.setPrice(formProduct.getPrice());
    // ... คัดลอกฟิลด์อื่นและอัปเดต ProductDetail ...
    return productRepository.save(existing);
}

public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}

private void linkDetail(Product product) {
    if (product.getDetail() != null) {
        product.setDetailWithBackReference(product.getDetail());
    }
}

private void cleanAndLinkReviews(Product product) {
    product.getReviews().removeIf(this::isBlankReview);
    product.getReviews().forEach(r -> {
        if (r.getReviewDate() == null) r.setReviewDate(LocalDate.now());
        r.setProduct(product);
    });
}
}

```

Constructor Injection: ProductService ประกาศ field private final ProductRepository productRepository; แล้วรับค่าผ่าน constructor เพียงจุดเดียว Spring จะมองเห็นว่าคลาสนี้ต้องการ ProductRepository และ inject Bean ที่ implement interface ดังกล่าวให้อัตโนมัติตอนสร้าง ProductService bean — ทำให้ ProductService ขึ้นกับ "interface" (DIP) ไม่ใช่ implementation ของ Spring Data JPA โดยตรง

เมธอดสำคัญ: saveNewProduct(...) เชื่อม back-reference ของทั้ง ProductDetail และ Review ก่อนส่งให้ repository บันทึก,
 updateProduct(...) โหลด entity เดิมจากฐานข้อมูลมาก่อนแล้วค่อยคัดลอกค่าจากฟอร์มทับ
 (ป้องกันข้อมูลอื่นที่ฟอร์มไม่ได้ส่งมาถูกเขียนทับด้วยค่า null), และ cleanAndLinkReviews(...) ที่ตัด Review
 ที่ผู้ใช้เว้นว่างในฟอร์มออกก่อนบันทึกจริง

ProductController.java

```

@Controller
@RequestMapping("/products")
public class ProductController {

    private final ProductService productService;

    // Constructor Injection
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public String list(Model model) {

```

```
model.addAttribute("products", productService.getAllProducts());
return "products/list";
}

@GetMapping("/add")
public String addForm(Model model) {
    Product product = new Product();
    product.setDetail(new ProductDetail());
    product.getReviews().add(new Review());
    model.addAttribute("product", product);
    return "products/add";
}

@PostMapping("/save")
public String save(@ModelAttribute("product") Product product,
                  RedirectAttributes redirectAttributes) {
    productService.saveNewProduct(product);
    redirectAttributes.addFlashAttribute("message",
        "เพิ่มสินค้า \"\" + product.getName() + "\" สำเร็จ");
    return "redirect:/products";
}

// editForm(), update(), deleteConfirm(), delete()
// ทำหน้าที่คล้ายกัน: รับ request -> เรียก service -> เลือก view/redirect
}
```

Constructor Injection: เช่นเดียวกับ Service, ProductController รับ ProductService ผ่าน constructor เพียงจุดเดียว ทำให้ Controller

ไม่รู้จักรายละเอียดภายในของ business logic หรือการ query ฐานข้อมูลเลย รู้แค่ว่ามี ProductService ที่เรียกใช้ได้ (SRP + DIP)

แต่ละเมธอดใน Controller มีรูปแบบเดียวกัน: รับ HTTP request → เรียก service เพื่องบรทัดเดียว → เลือกชื่อ View หรือ redirect กลับ

ไม่มี logic ทางธุรกิจปะปนอยู่เลยแม้แต่บรรทัดเดียว



Product Shop

Lab 8 — 1:1 & 1:N Relationship

เพิ่มสินค้า "iPhone 15 Pro (673380392-1 SEC 3)" สำเร็จ

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	iPhone 15 Pro (673380392-1 SEC 3)	Electronics	Apple	5	15000.00	MEMBER	13500.00	1 Year	1.61	1 รีวิว	แก้ไข ลบ

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380392-1 SEC 3)

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนในคลัง

5

ราคาปกติ (บาท)

25000.0

ประเภทส่วนลด

ส่วนลดสมาชิก (10%)

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

จอภาพ Super Retina XDR
 จอภาพ OLED ทั้งหมดจอ ขนาด 6.1 นิ้ว (แนวทแยง)
 ความละเอียด 2556 x 1179 พิกเซลที่ 460 ppi

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

1.61

ขนาด

71x147x7x171 mm

ประเทศที่ผลิต

China

บันทึกการแก้ไข

ยกเลิก



Product Shop

Lab 8 — 1:1 & 1:N Relationship

แก้ไขสินค้า "iPhone 15 Pro (673380392-1 SEC 3)" สำเร็จ

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	iPhone 15 Pro (673380392-1 SEC 3)	Electronics	Apple	5	25000.00	MEMBER	22500.00	1 Year	1.61	1 รีวิว	แก้ไข ลบ

